

Operating Systems – Study & Interview Notes

Compact notes covering the most important OS concepts for placement and technical interviews.

1. Introduction to Operating Systems

- **Operating System:** System software that manages computer hardware and provides services to application programs.
- **Main functions:** process management, memory management, file management, I/O management, security, networking, and resource allocation.
- **Kernel:** Core part of the OS that runs with high privileges and manages hardware resources.
- **System call:** Interface through which a user program requests a service from the kernel, such as `read()`, `write()`, `fork()`, or `open()`.
- **OS types:** batch, time-sharing, distributed, real-time, network, and embedded operating systems.

2. Processes and Threads

- **Process:** A program in execution. It has its own address space and resources.
- **PCB:** Process Control Block stores process state, program counter, CPU registers, scheduling information, memory information, and I/O status.
- **Process states:** new → ready → running → waiting/blocked → terminated.
- **Context switch:** Saving the state of one process/thread and loading another. It enables multitasking but adds overhead.
- **Thread:** Smallest unit of CPU execution within a process. Threads of the same process share code, data, and files but have separate stacks and registers.
- **Process vs thread:** Processes have separate address spaces and are relatively expensive to switch; threads are lighter and share process resources.

3. CPU Scheduling

- **Scheduling:** Selecting a ready process/thread for CPU execution.
- **Preemptive:** OS can interrupt a running process. **Non-preemptive:** running process keeps CPU until it terminates or blocks.
- **FCFS:** First Come First Serve; simple, non-preemptive, can cause convoy effect.
- **SJF:** Shortest Job First; minimizes average waiting time when burst lengths are known; normally non-preemptive.
- **SRTF:** Preemptive version of SJF.
- **Round Robin:** Each process gets a fixed time quantum. Good for interactive systems.
- **Priority scheduling:** Highest-priority process runs first; starvation can occur. **Aging** gradually increases priority of waiting processes.
- **Key metrics:** turnaround time = completion – arrival; waiting time = turnaround – burst; response time = first CPU start – arrival.

4. Process Synchronization

- **Race condition:** Result depends on the timing/order of concurrent accesses to shared data.
- **Critical section:** Code section that accesses shared resources.
- **Critical-section requirements:** mutual exclusion, progress, and bounded waiting.

- **Mutex:** Lock used to provide mutual exclusion; normally one owner at a time.
- **Semaphore:** Integer synchronization primitive manipulated using wait(P) and signal(V). Binary semaphores can act like locks; counting semaphores manage multiple identical resources.
- **Classical problems:** producer-consumer, readers-writers, and dining philosophers.

5. Deadlocks

- **Deadlock:** A set of processes is permanently waiting for resources/events held by each other.
- **Four necessary conditions:** mutual exclusion, hold and wait, no preemption, and circular wait.
- **Prevention:** design the system so at least one necessary condition cannot hold.
- **Avoidance:** allocate resources only when the resulting state remains safe; **Banker's algorithm** is a classic approach.
- **Detection:** allow deadlocks and periodically detect them, then recover.
- **Starvation:** A process waits indefinitely because resources/CPU repeatedly go to others; different from deadlock.

6. Memory Management

- **Logical/virtual address:** Address generated by CPU. **Physical address:** Actual RAM address.
- **Paging:** Divides virtual memory into fixed-size pages and physical memory into frames. Eliminates external fragmentation.
- **Page table:** Maps virtual pages to physical frames.
- **TLB:** Translation Lookaside Buffer caches recent page-table translations to speed address translation.
- **Segmentation:** Divides memory into variable-size logical segments such as code, data, and stack.
- **Fragmentation:** Internal fragmentation wastes space inside allocated blocks; external fragmentation leaves unusable gaps between allocations.

7. Virtual Memory

- **Virtual memory:** Allows programs to use an address space larger than physical RAM by keeping inactive pages on secondary storage.
- **Page fault:** Occurs when a referenced page is not currently in physical memory; OS loads it from storage.
- **Demand paging:** Loads pages only when they are needed.
- **Page replacement:** When no free frame exists, OS chooses a page to evict.
- **FIFO:** Replaces oldest page and can show Belady's anomaly.
- **LRU:** Replaces least recently used page.
- **Optimal:** Replaces the page whose next use is farthest in the future; theoretical benchmark because future references are unknown.
- **Thrashing:** Excessive paging leaves little time for useful CPU work.

8. File Systems and I/O

- **File:** Named collection of related data stored on secondary storage.
- **Directory:** Organizes files and stores metadata/references.
- **File allocation:** Contiguous, linked, and indexed allocation are common approaches.
- **Buffering:** Temporarily stores data during transfer between devices/processes.
- **Caching:** Keeps frequently accessed data in faster storage.

- **Spooling:** Queues data for a device that cannot serve all requests immediately, such as a printer.
- **DMA:** Direct Memory Access allows devices to transfer data to/from memory with limited CPU involvement.

9. OS Architecture & Booting

- **Monolithic kernel:** Most OS services run in kernel space; high performance but large trusted code base.
- **Microkernel:** Keeps minimal services in kernel space and moves many services to user space; improves modularity/isolation but can add communication overhead.
- **Boot process:** Firmware initializes hardware → bootloader loads kernel → kernel initializes subsystems → OS starts user-space services.
- **User mode vs kernel mode:** User programs have restricted privileges; kernel mode can execute privileged operations.

10. Common Interview Questions

- **Why is context switching expensive?** CPU state must be saved/restored and caches/TLBs may lose locality.
- **Why does paging reduce external fragmentation?** Fixed-size frames can be allocated to any free physical location, so contiguous physical space is not required.
- **Mutex vs semaphore?** A mutex is primarily ownership-based mutual exclusion; a semaphore is a signaling/resource-counting primitive.
- **Deadlock vs starvation?** Deadlock involves circular waiting and no progress among the involved processes; starvation means one process can wait indefinitely while others continue.
- **Why use virtual memory?** It provides process isolation, a larger logical address space, and efficient use of RAM through demand paging.
- **What is a system call?** A controlled entry point from user programs into kernel services.